

2^ο ΙΕΚ ΗΡΑΚΛΕΙΟΥ

ΜΑΘΗΜΑ: ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ (C++)

Φύλλο Ανάθεσης Εργασίας Αριθμ. 18

Κληρονομικότητα

Μια από τις πιο σημαντικές έννοιες του Αντικειμενοστραφούς Προγραμματισμού είναι η κληρονομικότητα, που μας επιτρέπει να ορίσουμε μια κλάση σε σχέση με μια άλλη κλάση. Με αυτό τον τρόπο η κλάση κληρονομεί χαρακτηριστικά από την κλάση από την οποία προκύπτει.

Δραστηριότητα 1 – Ιεραρχίες Κλάσεων

Ταξινομήστε τις παρακάτω οντότητες σε μια ιεραρχία, χρησιμοποιώντας το μηχανισμό της κληρονομικότητας:

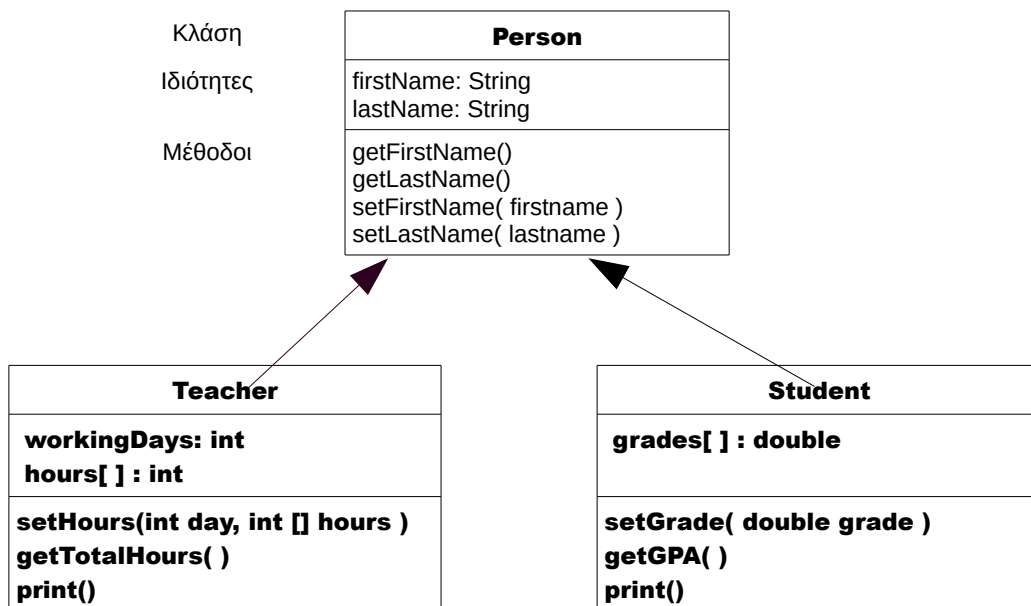
• Λαβράκι • Ψάρι • Παθολόγος • Θηλαστικό • Ελέφαντας	• Φώκια • Τσιπούρα • Άνθρωπος • Πίθηκος • Πουλί	• Εργαζόμενος • Γιατρός • Δικηγόρος • Ζωντανοί οργανισμοί • Οφθαλμίατρος
--	---	--

Δραστηριότητα 2 – Σχεδιασμός Κληρονομικότητας

Η κληρονομικότητα μας διευκολύνει στη δημιουργία και συντήρηση μιας εφαρμογής, υποστηρίζοντας την επαναχρησιμοποίηση κώδικα και γρήγορη υλοποίηση

Όταν ορίζουμε μια κλάση που απορρέει από μια άλλη κλάση δε χρειάζεται να ορίσουμε όλες τις ιδιότητες και τις μεθόδους της, αλλά να ορίσουμε μόνο τις ξεχωριστές ιδιότητες και μεθόδους της νέας κλάσης. Η νέα κλάση μπορεί να χρησιμοποιεί τις μεθόδους και τις ιδιότητες της κλάσης από την οποία απορρέει.

Στο παρακάτω διάγραμμα, έχοντας ορίσει την κλάση Person με τις ιδιότητες και τις μεθόδους της, μπορούμε στη συνέχεια να ορίσουμε δύο νέες κλάσεις (Teacher και Student) που θα κληρονομούν από την κλάση Person και επιπλέον θα ορίζουν τις δικές τους ιδιότητες και μεθόδους.



Δραστηριότητα 3 - Ορισμός Κληρονομικότητας

Στο παρακάτω παράδειγμα ορίζουμε τη κλάση Shape και στη συνέχεια ορίζουμε την κλάση Rectangle που κληρονομεί από τη Shape (Rectangle IsA Shape)

Δημιουργήστε ένα αρχείο με όνομα inheritance.cpp με το παρακάτω περιεχόμενο:

```
#include <iostream>

using namespace std;

// Base class
class Shape {
public:
    void setWidth(int w) {
        width = w;
    }
    void setHeight(int h) {
        height = h;
    }

protected:
    int width;
    int height;
};

// Derived class
class Rectangle: public Shape {
public:
    int getArea() {
        return (width * height);
    }
};

int main(void) {
    Rectangle Rect;

    Rect.setWidth(5);
    Rect.setHeight(7);

    // Print the area of the object.
    cout << "Total area: " << Rect.getArea() << endl;

    return 0;
}
```

Δραστηριότητα 4 - Ορισμός Πολλαπλής Κληρονομικότητας

Στο παρακάτω παράδειγμα ορίζουμε τις κλάσεις Shape και PaintCost στη συνέχεια ορίζουμε την κλάση Rectangle που κληρονομεί από τη Shape και από την PaintCost

Δημιουργήστε ένα αρχείο με όνομα multiple_inheritance.cpp με το παρακάτω περιεχόμενο:

```
#include <iostream>

using namespace std;

// Base class Shape
class Shape {
public:
    void setWidth(int w) {
        width = w;
    }
    void setHeight(int h) {
        height = h;
    }

protected:
    int width;
    int height;
};

// Base class PaintCost
class PaintCost {
public:
    int getCost(int area) {
        return area * 70;
    }
};

// Derived class
class Rectangle: public Shape, public PaintCost {
public:
    int getArea() {
        return (width * height);
    }
};

int main(void) {
    Rectangle Rect;
    int area;

    Rect.setWidth(5);
    Rect.setHeight(7);

    area = Rect.getArea();

    // Print the area of the object.
    cout << "Total area: " << Rect.getArea() << endl;

    // Print the total cost of painting
    cout << "Total paint cost: $" << Rect.getCost(area) << endl;

    return 0;
}
```